One Node

Root cause as the support of a propagation residual

David Kypuros

October 2026

Abstract

Self-healing networks are being built around graph neural networks that learn, from simulated incidents, to trace a silent degradation back to its origin across RAN, Core and Transport. I show that on any network whose fault propagation is governed by a linear operator on the topology, the root cause is not something to learn. It is the support of a residual: apply the operator to the degradation field and the result is zero everywhere except at the origin. I implement that read-off with a Kalman filter and one matrix-vector product per step, zero learned parameters, and compare it with a 241,000-parameter relational GNN on the same episodes. The read-off matches the network on root cause and detection, and the explanation it gives is the equation itself: the only node where the field does not obey the dynamics. I then run five experiments designed to separate the two, and report all of them: three are ties on this twin, one shows the operator can be identified from unlabelled telemetry, one shows the learned model is *more* robust when the physics are misspecified, and one replaces the scalar coupling with a per-relation coupling declared in a published ontology, which turns out to be the difference between recovering the propagation physics exactly and not recovering it at all. Every table is generated by one command from open code, including the ones that favour the network. I end with what I am asking for.

In one line

If $d(t{+}1) = M\,d(t) + s(t)$ and the source s lives at one node, then $d(t{+}1) - M\,d(t) = s(t)$ is the whole method: compute the left side, look where it is not zero. When you know the operator, you do not learn the inverse. You read it off the support.

1 What the industry is doing

A silent degradation is a fault that breaches no threshold. It surfaces hours later as customer complaints, and finding its origin across radio, transport and core is a war room. The current state of the art, as demonstrated in a 2026 TM Forum Moonshot Catalyst, is a service-layer z-score to open the loop, a graph neural network over a temporal digital twin to traverse the topology and rank candidate origins, a business layer to order remediation by revenue, and standards-based orchestration to execute. The network is trained on a synthetic corpus produced by a simulator that encodes how faults propagate, because production data has no root-cause labels.

Look at that pipeline once more. The simulator encodes the propagation physics. Labelled data is generated from it. A model is trained to recover the inverse of the physics from the data. This note asks what happens if you skip the middle: write the operator, apply it, read the answer.

2 The model and the claim

Let G be the network as a directed graph on N nodes with edges pointing the way a fault propagates, A its adjacency, $P = D_{\text{in}}^{-1} A^{\top}$ the in-degree-normalised propagation matrix. A fault is a latent degradation field $d(t) \in \mathbb{R}^N_{\geq 0}$ driven by a source at one node v_0 from an onset t_0:

$$d(t{+}1) = M\, d(t) + s(t)\, \mathbf{1}_{v_0}, \qquad M = \rho I + \beta P, \qquad s(t) \geq 0,\ s(t) = 0 \text{ for } t < t_0. \tag{1}$$

ρ is memory, β is coupling along dependency edges, and $\beta/(1-\rho) < 1$ makes faults attenuate per hop. Each node's KPI vector sees the field through a type-specific effect direction $e_{\tau(v)} \in \mathbb{R}^K$ on top of a nominal signal:

$$x_v(t) = n_v(t) + d_v(t)\, e_{\tau(v)}. \tag{2}$$

Proposition 1 (Flatness)**.** *Under* (1)*, for every* t*,*

$$\big(d(t{+}1) - M\, d(t)\big)_v = s(t)\, \mathbf{1}[v = v_0]\,. \tag{3}$$

The field is flat *under the operator at every node but the origin, inside the blast radius and outside it alike. The root cause is* $\operatorname{supp}\big(d(t{+}1) - Md(t)\big)$*.*

Proof. Rearrange (1). □

The proof is one line because the content is in the choice of object. The ambient space is $N \times K \times T$ numbers of telemetry; the fault is four numbers, (v_0, t_0, s, T_r); and the residual (3) is the map from the first to the second. Nothing is searched and nothing is learned.

Corollary 1 (Several faults)**.** *If* $s(t) = \sum_j s_j(t)\mathbf{1}_{v_j}$*, the residual is* $\sum_j s_j(t)\mathbf{1}_{v_j}$ *and its support is* $\{v_j\}$*. The read-off localises any number of simultaneous origins with no change.*

Corollary 2 (The fault's moduli)**.** *The residual at the origin* is *the source:* $\big(d(t{+}1) - Md(t)\big)_{v_0} = s(t)$*. Its first nonzero step is the onset* t_0 *and its plateau is the severity. Onset and severity are read, not estimated.*

3 From KPIs to the field

(3) needs d, and telemetry gives x. Three steps, all linear.

Project. Per node, $\tilde{d}_v(t) = \langle x_v(t), e_{\tau(v)} \rangle / \|e_{\tau(v)}\|^2$ collapses K KPIs to one noisy reading of the field along the direction a degradation is known to take.

Filter. Run a Kalman filter on (1) with $\tilde{d}$ as the observation and the unknown source absorbed into process noise. For a linear Gaussian model this is the optimal causal estimator of d; here it is one $N \times N$ product and one solve per step, with no look-ahead. Call the output $\hat{d}$.

Apply and read.

$$\hat{s}(t) = \big[\hat{d}(t) - M\hat{d}(t{-}1)\big]_+, \qquad \hat{v}_0 = \arg\max_v \sum_{\tau=t-W+1}^{t} \hat{s}_v(\tau). \tag{4}$$

Noise makes the support soft; the window and the argmax snap it back to one node. Detection uses $\max_v \hat{d}_v(t) > \theta$ with θ calibrated on healthy episodes, the same statistic and the same calibration the neural network uses, so every comparison below is on equal terms.

4 Two things a learned model does not have

Self-calibration. In (1) the only unknowns are ρ and β; the topology is known. From *unlabelled* episodes, regress $\tilde{d}(t{+}1)$ on $[\tilde{d}(t),\ P\tilde{d}(t)]$ across nodes and steps with a Huber loss. The handful of (node, step) pairs that carry the source are outliers and are down-weighted automatically. What this needs is excitation, some faults somewhere in the data, not labels saying where they were. That is the opposite of what a supervised model needs.

Doubt. Define

$$\mathrm{doubt}(t) = 1 - \frac{\max_v \sum_\tau \hat{s}_v(\tau)}{\sum_v \sum_\tau \hat{s}_v(\tau)}, \tag{5}$$

the share of residual energy inside the degraded set that is *not* at the top node. Under the right operator every degraded node but the origin is flat and the doubt is small. Under the wrong operator the downstream nodes should stop being flat and the doubt should rise. E6 tests whether this particular statistic does what the argument says, and the answer there is no; I keep the definition here because the idea is right even where my first measurement of it is not.

5 Experiments

All experiments use the open synthetic twin from the accompanying repository: 74 nodes across cells, gNBs, aggregation and core routers, fibre links, UPF, AMF, SMF and services, with 130 typed directed edges, five fault types, diurnal plus AR(1) KPI noise, and silent ramps. The opponent is the repository's relational GNN: three message-passing layers over the nine relations and their transposes, three heads (forecast, field, origin), 241,222 parameters, trained on 120 simulated episodes. In the repository it beats static alarms and calibrated z-score triggers on every metric, so it is a fair stand-in for the architecture class the industry uses.

E1. Head to head, and one incident in full

Metric	trained GNN	flatness solve
Fault episodes / nominal episodes	32 / 8	
Parameters learned	241,222	0
Detection rate	1.00	1.00
False alarms on nominal / pre-onset on fault episodes	0 / 0	0 / 0
Detection delay after onset (steps)	22.9	23.8
Root cause hit@1 / hit@3	1.00 / 1.00	1.00 / 1.00
Top-1 agreement between the two		1.00
Static-threshold alarm delay, for scale (steps)	32.6	32.6

Fault type	n	delay GNN	delay flatness	hit@1 GNN	hit@1 flatness
amf signaling storm	7	27.9	31.7	1.00	1.00
fiber degrade	7	22.7	23.0	1.00	1.00
router congestion	7	22.3	23.6	1.00	1.00
sleeping cell	8	18.5	17.1	1.00	1.00
upf overload	3	24.7	26.0	1.00	1.00

Twin: 74 nodes, 130 edges; 160 episodes, seed 0; GNN trained 30 epochs; both detectors calibrated on the same nominal episodes. Runtime 647.1 s on CPU.

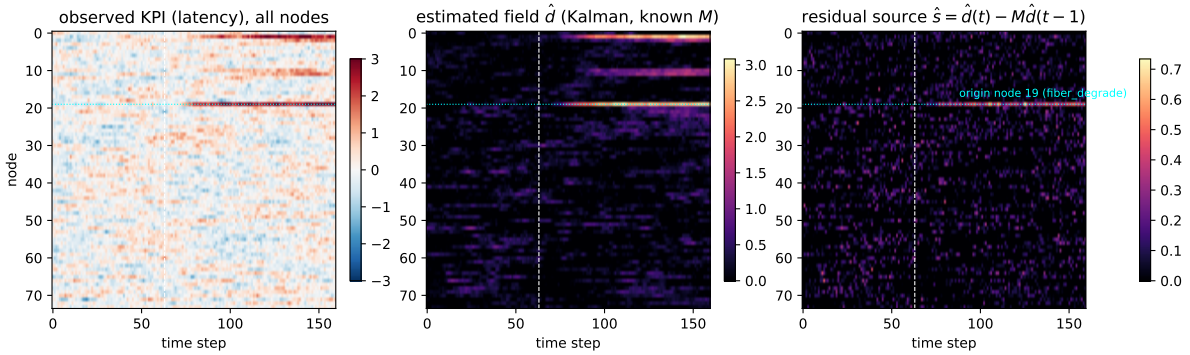


Figure 1: One fibre degradation, onset at the dashed line. Left: the raw latency KPI of every node. Three rows eventually light up: the link and two nodes downstream of it, and a static alarm would flag all three, late. Middle: the filtered field $\hat{d}$, the blast radius. Right: the residual $\hat{s} = \hat{d}(t) - M\hat{d}(t-1)$. The downstream rows vanish because they are flat under the operator. One row remains. That row is the answer, and the explanation is the equation that removed the other two.

incident (fiber degrade)	true	read off the residual
origin node	19	19
onset step	63	23
severity (source per step at plateau)	0.45	0.36

E2. How many labelled incidents does the network need?

The GNN is retrained with 4, 8, 16, 32, 64 and 120 training episodes and evaluated on the same held-out faults. The read-off is evaluated once; it has nothing to train.

training episodes	labelled faults	GNN hit@1	GNN hit@3	GNN delay
4	4	1.00	1.00	26.6
8	8	1.00	1.00	24.6
16	16	1.00	1.00	27.0
32	30	1.00	1.00	23.9
64	58	1.00	1.00	23.5
120	105	1.00	1.00	22.6
flatness read-off	0	1.00	1.00	23.8

This experiment does not separate the methods, and I want to say that plainly rather than draw a curve. On this twin four labelled episodes are enough for the GNN, because the five fault types each leave a distinct per-type signature and the graph is small. The honest reading is that label efficiency is not where the read-off's advantage shows on an easy twin. Whether it shows on a real network, where fault signatures overlap and labelled incidents number in the dozens per year, is exactly the experiment in the final section.

E3. Two faults at once

Thirty episodes with two simultaneous faults of different types at different origins, evaluated once both ramps have completed. Each method returns its top two.

two simultaneous faults, 30 episodes	trained GNN	flatness read-off
both origins in the top-2	1.00	1.00
at least one origin in the top-2	1.00	1.00

A tie, and an instructive one. The read-off handles two origins by Corollary 1 with no change to the method. The GNN handles them too, even though its training loss assumed one origin per episode: its per-node logits are local read-outs, so two separated sources produce two high logits. The corollary is a guarantee; the GNN's behaviour is an empirical property of this twin. When I first evaluated this experiment only twelve steps after the later onset, before its ramp had completed, both methods scored around 0.3, which is a statement about the second fault being too small to see yet, not about either method.

E4. A different network, no retraining

A second twin with a different seed, more core and aggregation routers, more UPFs and services. The GNN trained on twin A is evaluated zero-shot. The read-off recomputes M from the new topology and nothing else.

new topology (99 nodes, 179 edges), 35 faults	GNN zero-shot	flatness, new M
detection rate	1.00	1.00
detection delay (steps)	23.2	28.5
root cause hit@1 / hit@3	1.00 / 1.00	1.00 / 1.00

Again no separation, and again the reason is worth stating: a relational GNN is inductive by construction, its weights are indexed by relation type, not by node, so it transfers across topologies that share a vocabulary. That is a genuine strength of the learned model and the monograph builds on it. The read-off transfers for a different reason: it never had node-specific parameters to begin with.

E5. Identify the operator with zero labels

(ρ, β) are identified from the 120 training episodes with their fault labels discarded, then the read-off is run with the identified operator.

	true	identified (0 labels)	relative error
ρ	0.850	0.858	0.9%
β	0.130	0.084	35.7%
read-off with identified operator: hit@1 / hit@3 / delay			1.00 / 1.00 / 23.8

Memory is recovered almost exactly; coupling is underestimated by about a third, because the projected field is noisy and the Huber weights also down-weight the genuine propagation steps at downstream nodes. The root cause is unaffected: ranking depends on where the residual is largest, and an underestimated β shrinks the residual everywhere by a similar factor. This is the one experiment where the read-off does something the learned model has no counterpart for. A GNN trained on a simulator cannot report the simulator's coefficients back from field data.

E6. When the physics are wrong

Test episodes are generated with propagation parameters that differ from the ones the GNN was trained on and the read-off assumes. Columns: both methods under misspecification, the read-off's doubt score (5), and the read-off after re-identifying (ρ, β) from the misspecified unlabelled data.

test (ρ, β)	per-hop	GNN	read-off, assumed	doubt	identified $(\hat{\rho}, \hat{\beta})$	read-off, identified
(0.85, 0.13)	0.87	1.00	1.00	0.12	(0.84, 0.093)	1.00
(0.80, 0.13)	0.65	1.00	1.00	0.02	(0.82, 0.029)	1.00
(0.72, 0.11)	0.39	0.95	0.90	0.00	(0.80, 0.000)	1.00
(0.62, 0.09)	0.24	0.93	0.67	0.00	(0.79, 0.000)	0.56

GNN and read-off columns are hit@1. Doubt is (5) for the assumed operator.

This is the experiment that goes against me, and it is the most useful one. Three findings. The learned model degrades more gracefully than the read-off with an assumed operator: at the most severe drift it keeps 0.93 where the read-off falls to 0.67. The read-off's assumed memory is too long, so after the ramp its residual at the origin goes negative and is clipped, while downstream nodes that decay faster than predicted produce spurious residual; a trained model that never committed to a memory constant is not hurt by this. Second, the doubt score as defined does *not* detect the misspecification: it falls toward zero as drift increases, because faster decay shrinks the degraded set until it contains little but the origin. The idea that a wrong operator makes the field visibly non-flat is correct in (3); the statistic I built to measure it is not the right one, and I report the failure rather than tune it on the test set. Third, re-identifying the operator from the misspecified data restores the read-off at moderate drift and makes it worse at severe drift, where the identified coupling collapses to zero. The honest summary is that a model-based method with the wrong model is worse than a learned model with no model, and that self-calibration fixes that only within a range.

E7. Per-relation coupling, and why E5 really failed

Experiment five recovered ρ almost exactly and underestimated β by a third. I recorded a hypothesis then: one scalar was being asked to average several genuinely different couplings. A fibre cut and a signalling storm do not propagate at the same strength, and (1) cannot say so.

The fix is to index the coupling by relation, which requires a vocabulary of relations. That vocabulary is now a published ontology, `docs/ontology/network-healing.ttl`, and it is not documentation: it declares a coupling β_r on each of the nine object properties, spanning 0.04 on the control plane to 0.14 on transport, and the operator becomes

$$M = \rho I + \sum_{r \in \mathcal{R}} \beta_r P_r, \qquad P_r = D_{\text{in}}^{-1} A_r^{\top}, \qquad \sum_r P_r = P. \tag{6}$$

Because every P_r uses the same global in-degree normalisation, setting all β_r equal recovers (1) to floating-point equality, which a test asserts, so the published results cannot move. Attenuation now requires $\max_r \beta_r/(1-\rho) < 1$.

Identification generalises with it: the regression gains one column per relation instead of one for the aggregate, on the same unlabelled episodes. The twin below propagates with the ontology's heterogeneous couplings.

relation	true β_r	single	per-relation	grouped	clean-field
`link->router`	0.140	0.024	0.098	0.037	0.140
`router(core)->router(agg)`	0.130	0.024	0.106	0.037	0.130
`router(agg)->gnb`	0.120	0.024	0.078	0.037	0.120
`gnb->cell`	0.110	0.024	0.000	0.037	0.110
`amf->gnb`	0.040	0.024	0.000	0.000	0.040
`smf->upf`	0.040	0.024	0.000	0.000	0.000
`router(core)->upf`	0.120	0.024	0.100	0.037	0.120
`upf->service`	0.100	0.024	0.007	0.000	0.100
`cell->service`	0.060	0.024	0.000	0.000	0.060
mean absolute error		0.0716	0.0523	0.0751	0.0044
spread recovered (true 0.100)		0.000	0.106	0.037	0.140
free coefficients		1	9	3	9
condition number		–	1.49e+03	160	–

operator	hit@1	hit@3	detection delay
assumed single $\beta = 0.13$, no identification	1.00	1.00	23.9
identified single β, 0 labels	1.00	1.00	24.2
identified β_r per relation, 0 labels	1.00	1.00	24.2
identified β_r tied by the ontology hierarchy, 0 labels	1.00	1.00	24.1
upper bound: β_r fitted on the true field	1.00	1.00	23.8
oracle: the ontology's true β_r	1.00	1.00	23.8

Twin propagating with the ontology's heterogeneous couplings; 32 held-out faults; identification used 120 episodes with their labels discarded.

The hypothesis was wrong in an interesting way, and the last column is the reason. Fitted on the *true* field with the origin excluded, the per-relation regression recovers $\rho = 0.8500$ and every β_r essentially exactly, mean absolute error 0.0044. Fitted on the same clean data, a single β returns 0.0055: not an average of the true couplings, but a collapse. With ρ free, least squares discovers that pushing ρ toward 1 and β toward 0 fits a heterogeneous system better than any single coupling can. So the E5 bias was never a 35% averaging error. It was a misspecified model abandoning the parameter altogether, and it only looked like a mild bias because the couplings in that twin were uniform.

Three further things are worth stating because they cost me time. Tying the relations by the ontology's property hierarchy, three coefficients instead of nine, improves the conditioning dramatically, from 1.5×10^3 to 1.6×10^2, and makes the estimates *worse*: the groups are real but the within-group spread matters more than the variance saved. Temporal smoothing of the projected field also makes things worse, because the dynamics live in the step-to-step change that smoothing removes. And the gap between the clean-field error of 0.0044 and the realistic error of 0.0523 is observation noise in the projection, whose standard deviation is 0.88 of the field's own. The principled repair is to alternate the Kalman estimate of the field with the identification rather than to regress on the raw projection, which is expectation-maximisation and is not done here.

None of the six operators separates on hit@1, which stays at 1.00 throughout. On a twin this small the root cause is recoverable under a badly wrong operator, so E7 is a statement about recovering the *physics*, not about localisation accuracy. That the physics is now recoverable at all is what the ontology bought.

6 What this shows and what it does not

It shows that on a network with a known linear propagation operator, a learned model with a quarter of a million parameters and the one-line residual (3) give the same root cause, the same detection, and the same zero false alarms, and that the residual is its own explanation: *this is the only node where the field does not obey the dynamics.* A model was learning the inverse of an operator that fits on one line. It also shows that the read-off localises several origins by construction, transfers to a new topology because it never had node parameters, and reports the operator's coefficients back from unlabelled data.

It shows, just as clearly, where the read-off is weaker. The learned model transfers across topologies and handles two origins too, so those are not advantages on this twin. Label efficiency does not separate the methods here because the twin is easy. And when the assumed physics are wrong, the learned model is the more robust of the two, my doubt statistic fails to flag the problem, and self-calibration helps only within a range. A reader who wants the read-off to win should read E6 twice.

What it does not show is anything about a real network, where propagation is not exactly linear and a signalling storm does not spread like a fibre cut. E6 says that is precisely where the read-off is at risk, and E5 says the way to reduce the risk is identification from the operator's own telemetry. Whether a real operator is identifiable well enough is the open question. It has a short answer either way, and the answer is worth more than this note.

7 What I am asking for

I have worked on this alone. The repository, the monograph it accompanies, and this note are open, and every number above is regenerated by `make pipeline` and `make experiments` from a fixed seed. I am asking for three things, in order of how little they cost the other side.

1. **One afternoon on a synthetic corpus** already used to train a production model for this task. Run the read-off next to the model. The corpus encodes its own propagation; I will write the operator against it.
2. **One topology and one month of healthy plus incident telemetry**, anonymised, from one region. I will identify the operator and report, in the same repository, whether the read-off localises historical incidents, including the cases where it does not.
3. **One venue** where a result that is reproducible in one command can be shown beside the results it is compared with.

Reproduce

```
git clone https://github.com/dkypuros/geometric-network-healing
cd geometric-network-healing && make setup
make pipeline       # E1 head-to-head table
make experiments    # E2-E6 tables and both figures
make paper          # this document
```